



Universidad
Nacional
de Quilmes

Informe “Unq-Shop”

Profesores:

- Diego Cano
- Matias Urbieta
- Matias Butti

Integrantes:

- Franco Villamayor
- Rodrigo Bornia

Emails:

- francojavier.villamayor@gmail.com
- rodrigobornia@gmail.com

Decisiones de diseño

Para una mejor organización decimos estructurar el proyecto en diferentes packages de la siguiente manera.

Criterio:

Se modela la clase *Criterio* la cual se encarga de gestionar el criterio de búsqueda de paquetes y productos de manera simple (por nombre , por precio máximo, por categoría, por disponibilidad) o por Criterios Complejos que para eso usamos el patrón [Composite](#) para poder combinar estos operadores lógicos (and , or , not), donde las hojas serían los criterios simples y los compuestos serían los criterios complejos.

Ecommerce:

Se presenta la clase *Ecommerce* donde se gestiona la búsqueda de los productos a partir de un criterio (sea una regla simple o un Composite complejo), se lo aplica al catálogo completo y devuelve los resultados. También genera un reporte primero realiza el "Double Dispatch" recorriendo el catálogo y permitiendo que el visitante entre a cada ítem a recolectar datos (item.accept(visitor)). Luego, recupera los resultados y le cede la responsabilidad del formato visual a la estrategia de exportación elegida.

Envío:

En este paquete se encuentra la clase *MetodoDeEnvio* (Estrategia) donde usamos el patrón [Strategy](#) para poder intercambiar cómo calcular el costo de distintas maneras:

Contrato Base (Estrategia): La clase abstracta MetodoDeEnvio define el comportamiento obligatorio: calcula el costo del envío y estimar la fecha de entrega.

Implementaciones Específicas (Estrategias Concretas): Las clases EnvioEstandar, EnvioExpress y RetiroSucursal resuelven estos cálculos basándose en sus propias reglas de negocio. Por ejemplo, el retiro en sucursal es gratuito y su demora depende del stock local, mientras que las otras opciones varían según el peso o el valor del pedido.

Para los envíos a domicilio, las clases usan interfaces de servicios externos (CorreoArgentino y LibEnvioExpress).

Método de pago:

Esta clase gestiona distintas formas de cobro. Se utilizó el Patrón [Template Method](#), el cual define el esqueleto de un algoritmo en una clase base y delega los detalles de cada paso a sus subclases:

Esqueleto del Algoritmo (Clase Base): La clase abstracta MetodoDePago define el método orquestador procesarPago(). Este es el método obligatorio que todo pago debe seguir: validar datos, reservar fondos, ejecutar la transacción y notificar el resultado.

Implementaciones Específicas (Subclases): Las clases BilleteraVirtual, TarjetaDeCredito y TransferenciaBancaria heredan este esqueleto y se fijan que hacer en cada paso conectándose con sus APIs externas. El metodo Hook es reservarFondos() se deja vacío intencionalmente porque no aplica a la naturaleza de ese medio de pago.

Pedido:

En este paquete se gestiona el ciclo de vida completo de una compra, desde que se seleccionan los productos hasta que llegan al cliente. Para controlar las transiciones de forma segura y automatizar las acciones derivadas, se combinaron los patrones [State](#) y [Observer](#).

El patrón State forma parte del diseño para los estados del Pedido, dónde Pedido es el contexto, el estado una clase abstracta y los estados del Pedido son las subclases de Estados concretos.

Luego el patrón Observer se ve reflejado en el diseño de las notificaciones que hace el pedido, dónde Pedido es el Sujeto que tiene observadores (NotificadorEmail, Fidelización, Facturación) que están a la expectativa de los cambios que le notifica el pedido para actuar de la manera que le corresponde a cada uno.

Producto:

En esta clase se modeló el comportamiento de los artículos que se pueden vender, dividiéndolos conceptualmente en dos tipos: **productos base** y **paquetes** (conjuntos de productos). Para lograr esto, se utilizó una clase abstracta padre y dos subclases hijas, aplicando herencia, polimorfismo y los patrones de diseño [Composite](#), donde la hoja es Producto base y el compuesto Paquete.

Visitor:

Para lograr esto de manera escalable y sin alterar el código de los artículos, se combinaron los patrones [Visitor](#), [Strategy](#) dividiendo el proceso en dos etapas estructurales:

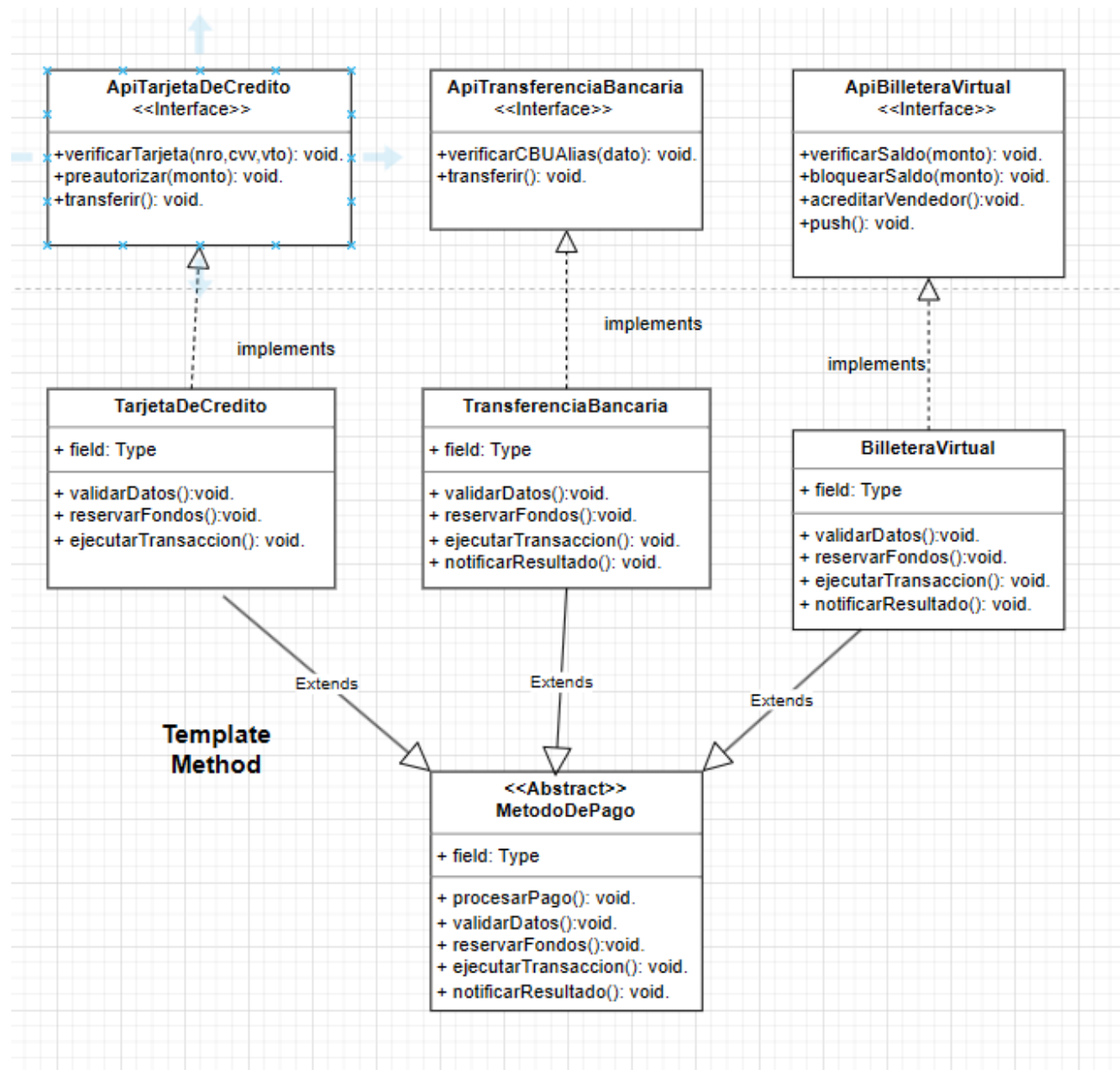
Recolección de Datos (Visitor): A través del `RecolectorVentasVisitor`, el sistema "visita" de forma externa los productos y paquetes, cruzando su información con el historial de pedidos en un rango de fechas. Calcula las unidades vendidas y el precio promedio, y empaqueta estos resultados en objetos ligeros de transporte (`ItemReporteDTO`).

(Strategy): Una vez recolectados los datos, el sistema delega la presentación visual a distintas estrategias de exportación (`ExportadorCSV`, `ExportadorHTML`, `ExportadorTXT`). Cada clase define cómo se estructura el texto final.

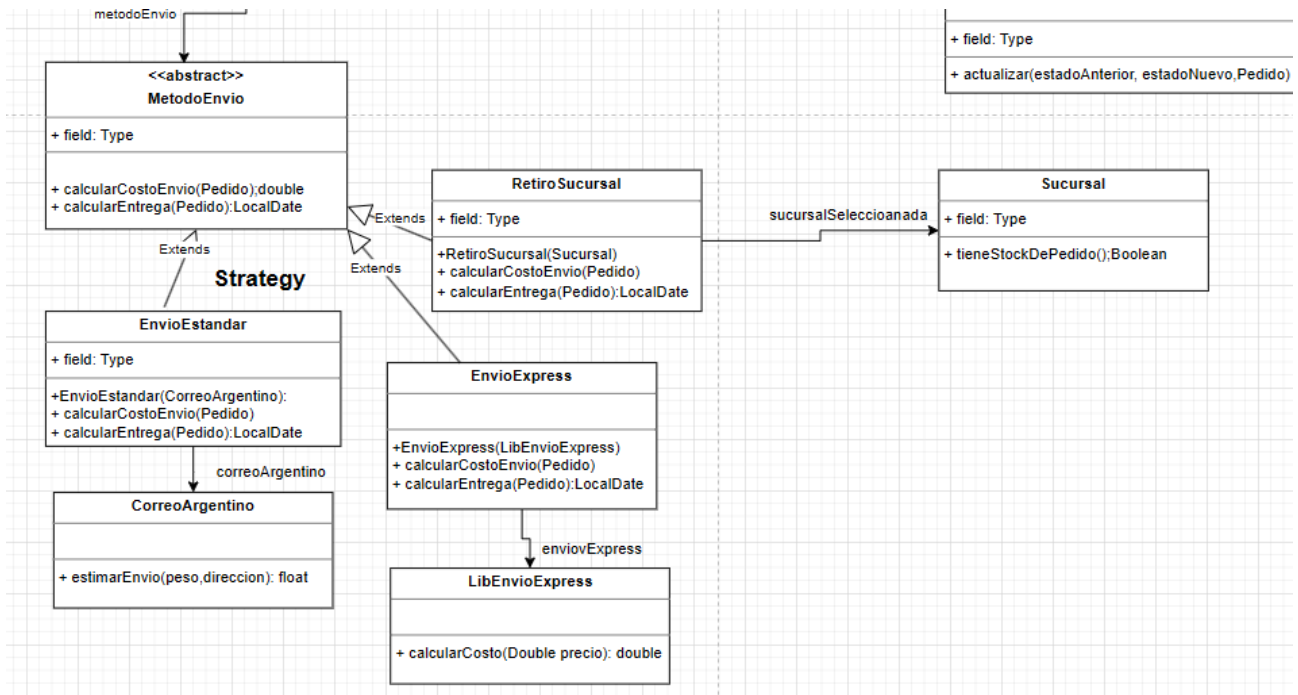
PATRONES UTILIZADOS

Se presentan los patrones aplicados en cada clase y sus respectivas partes según el libro de Gamma.

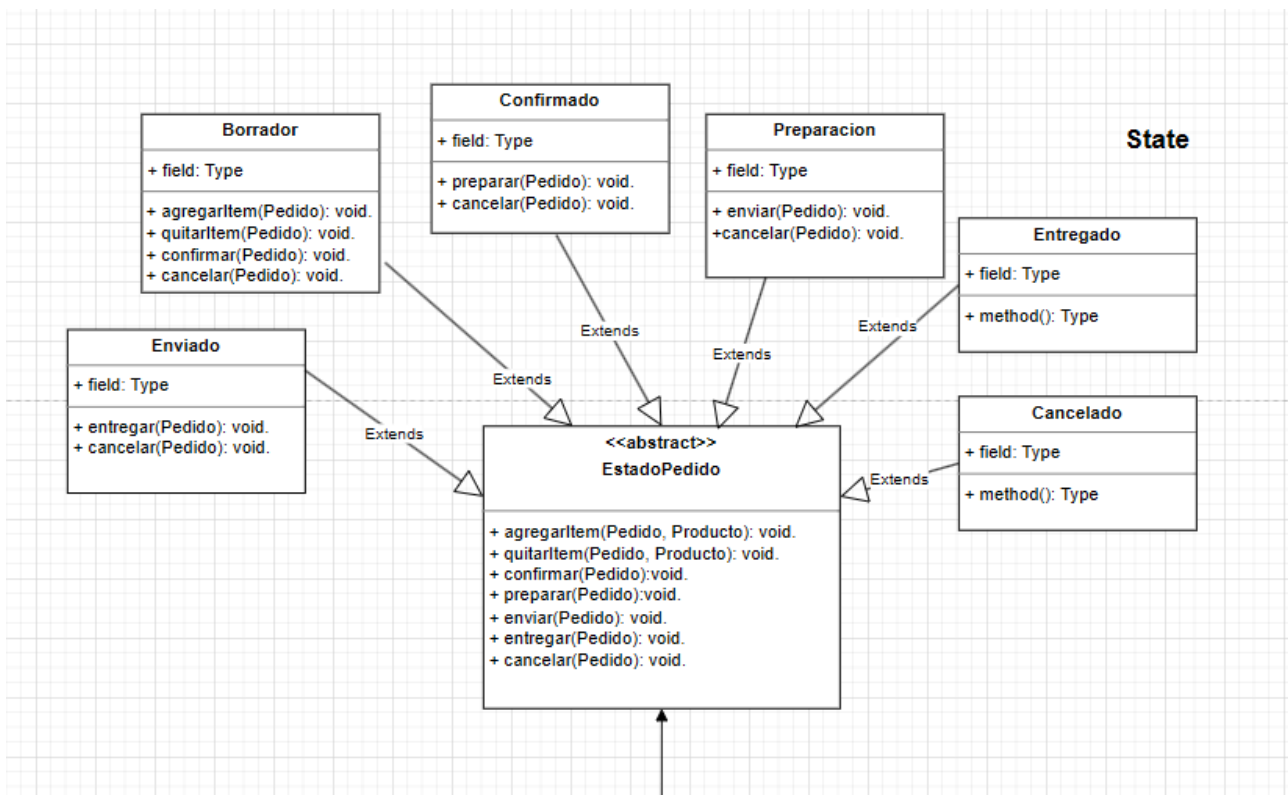
Patron Template Method:



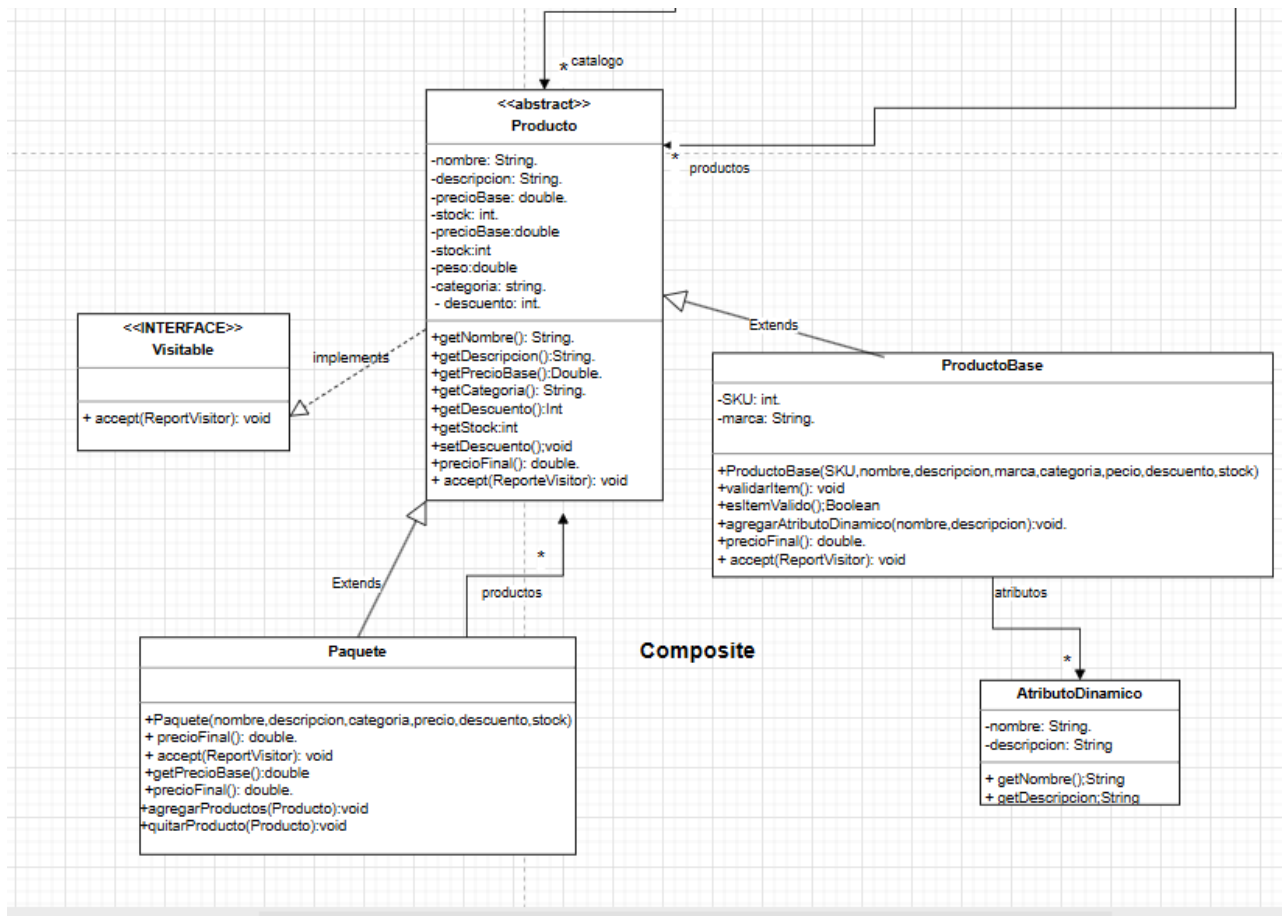
Patrón Strategy:

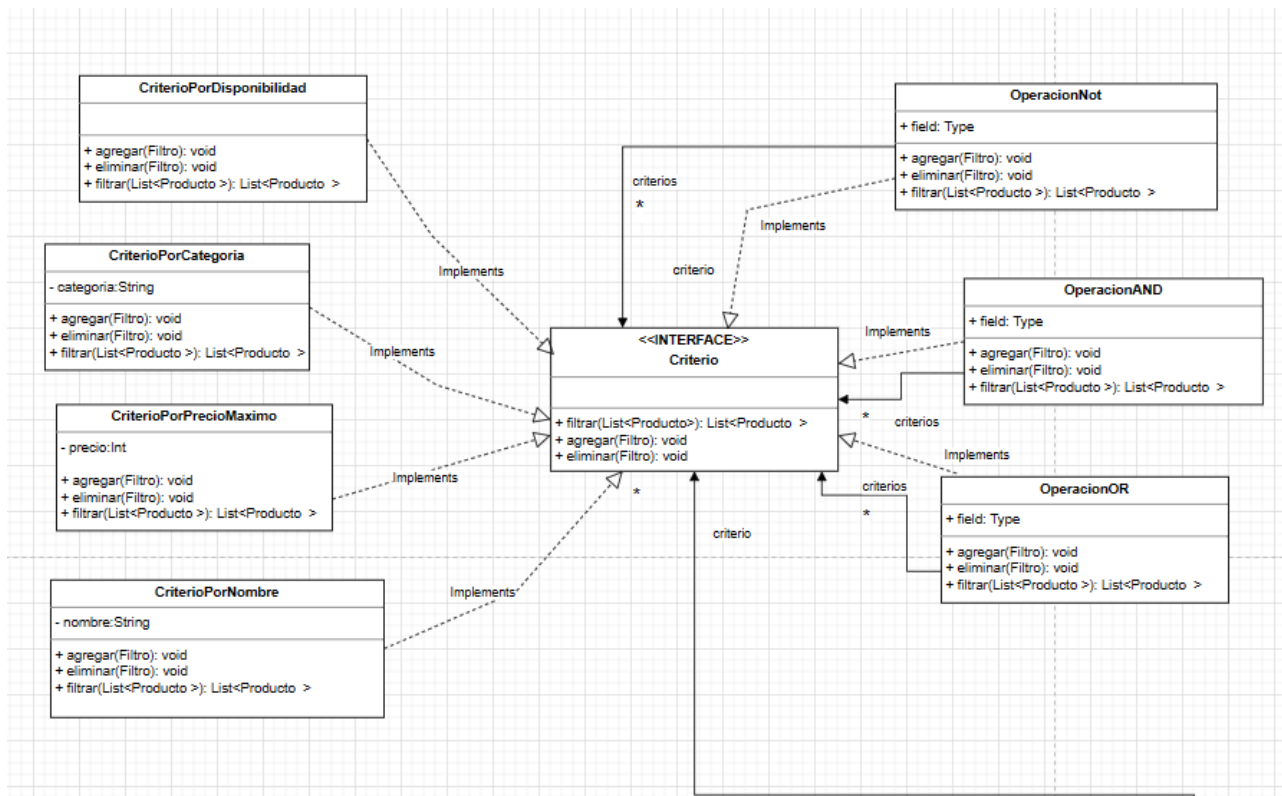


Patron State:

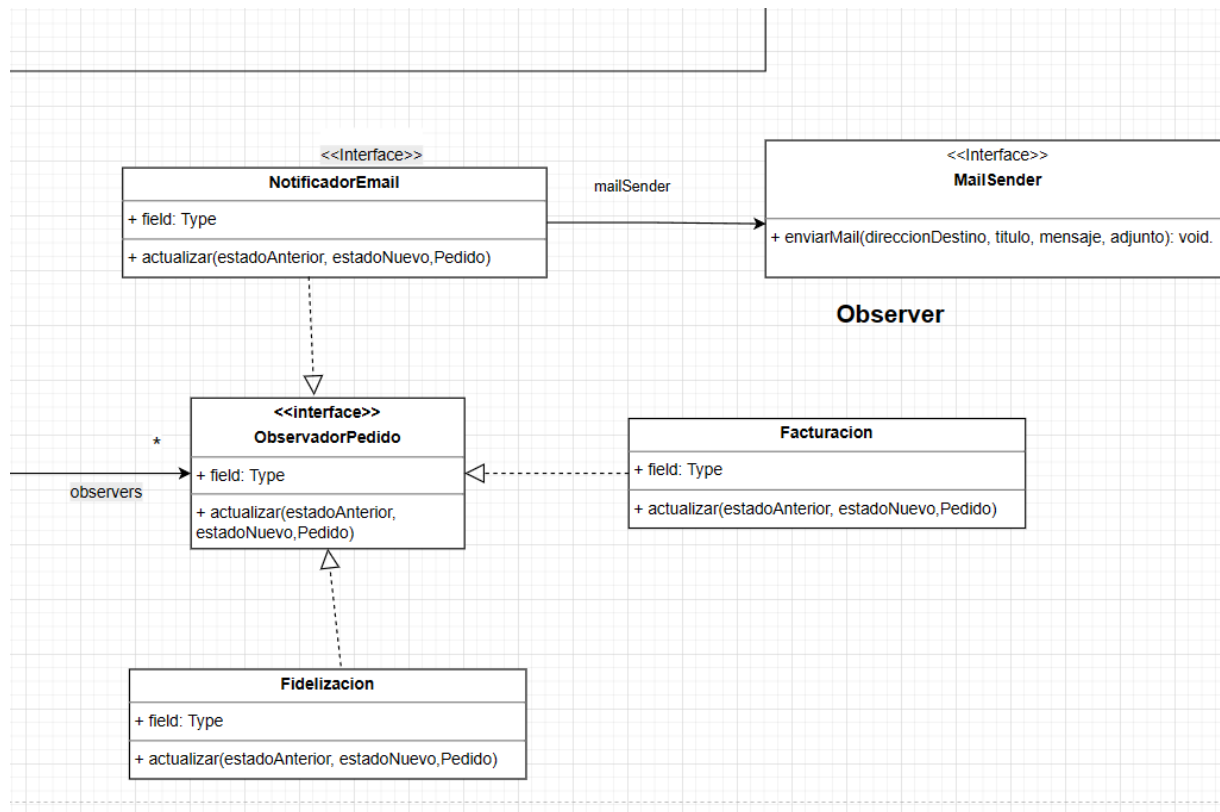


Patron Composite:





Patrón Observer:



Patrón Visitor y Strategy:

